

Análisis de Personas Desaparecidas en Perú — RENIPED

Entrega Parcial

Integrantes:

- [Jose Carlos Salinas] — [jc.salinasma@alum.up.edu.pe]
- [Arturo Alvarez de la Torre] — [a.alvarezdelatorres@alum.up.edu.pe]
- [Gianella Mayra Silvestre Nina] — [gm.silvestren@alum.up.edu.pe]
- [Jose Luis Atto Pintado] — [jl.attop@alum.up.edu.pe]
- [Alyssa Antuanette Trujillo Cruzado] — [aa.trujillocr@alum.up.edu.pe]

El objetivo de este proyecto es desarrollar un dashboard exploratorio en Streamlit para analizar patrones demográficos, temporales, geográficos y textuales en reportes de personas desaparecidas en Perú, utilizando datos del sistema RENIPED.

1. Dataset: RENIPED

El **Registro Nacional de Información de Personas Desaparecidas (RENIPED)** es el sistema oficial de la Policía Nacional del Perú para el registro y seguimiento de denuncias de desaparición a nivel nacional. Los datos son de acceso público a través del portal institucional (<https://desaparecidosenperu.policia.gob.pe/>) y no tienen restricción de uso para fines informativos o académicos.

El dataset utilizado fue publicado por Aybar-Flores et al. (2026), "*Uncovering Disappearance Dynamics in Peru: A Two-Stage Multimodal Framework for Behavioral Segmentation and Voluntariness Prediction in Missing-Person Reports*", Springer Nature Switzerland. DOI: https://doi.org/10.1007/978-3-032-20752-4_1. El dataset y el código asociado están disponibles en <https://github.com/yoce3/MissingPersonAnalysis>. Cada registro corresponde a una denuncia formal de desaparición e incluye información sobre la persona desaparecida, las circunstancias del hecho, características físicas, ubicación geográfica y el estado de resolución del caso.

Las variables utilizadas en el análisis se agrupan en las siguientes categorías:

Categoría	Variables
Demográficas	EDAD, PAIS DE NACIMIENTO
Temporales	Fecha Hecho, Fecha Denuncia, Horas para Denunciar, Horas para Aparecer
Geográficas	Lugar Hecho, Latitud, Longitud
Estado del caso	Aparecido, Fecha de Aparición
Características físicas	Tez, Fenotipo, Cabello, Contextura, Estatura, Ojos, Sangre
Texto libre	Circunstancias, Vestimenta, Otras observaciones

2. Procesamiento y limpieza

El pipeline de procesamiento se implementó en `src/processing.py` con las funciones `load_data()` y `clean_data()`. Se aplicaron los siguientes pasos:

2.1 Extracción de región

La variable `Lugar Hecho` contiene la dirección completa de la desaparición concatenada con guiones:

```
LIMA-LIMA-MIRAFLORES- AV. LARCO 345
```

Para el análisis geográfico agregado se extrajo el primer segmento como variable `region`:

```
df["region"] = df["Lugar Hecho"].str.split("-").str[0].str.strip().str.title()
```

Esto permitió agrupar los casos a nivel de departamento, revelando que Lima concentra el 36.3% del total de registros, seguida por Cusco (6.8%) y Arequipa (5.3%).

2.2 Coordenadas fuera del territorio peruano

Al revisar los campos `Latitud` y `Longitud` se identificaron **4 registros** con coordenadas geográficamente imposibles para el Perú (latitudes positivas o longitudes fuera del rango continental). Estos errores corresponden a fallas en el geocodificado de la dirección de origen.

Se aplicó un bounding box geográfico y se anularon únicamente las coordenadas de los registros afectados, conservando el resto de su información:

```
PERU_LAT = (-18.5, -0.5)
PERU_LON = (-81.5, -68.5)

bad_coords = (df["Latitud"] < PERU_LAT[0]) | (df["Latitud"] > PERU_LAT[1]) | \
              (df["Longitud"] < PERU_LON[0]) | (df["Longitud"] > PERU_LON[1])

df.loc[bad_coords, ["Latitud", "Longitud"]] = np.nan
```

2.3 Valores negativos en Horas para Aparecer

La variable `Horas para Aparecer` debería contener únicamente valores positivos, representando el tiempo transcurrido desde la desaparición hasta la reaparición. Se encontraron **109 registros** con valores negativos, producto de inconsistencias en el orden de las fechas registradas en el sistema fuente.

Dado que el resto de los campos de estos registros son válidos, se optó por anular solo el campo afectado en lugar de eliminar la fila completa:

```
df.loc[df["Horas para Aparecer"] < 0, "Horas para Aparecer"] = np.nan
```

2.4 Duplicados y estandarización

Se eliminaron **16 registros duplicados** detectados tras la carga. Las variables de características físicas (**Tez**, **Fenotipo**, **Cabello**, **Contextura**, entre otras) se estandarizaron a formato título, y las entradas con valor **"Sin Información"** fueron tratadas como datos faltantes.

2.5 Validación de columnas

`load_data()` valida que el CSV exista y tenga las columnas mínimas esperadas antes de transformar nada (`REQUIRED_RAW_COLUMNS`). `clean_data()` y `get_summary()` hacen lo mismo con las columnas que necesitan. Esto se centralizó en una función `require_columns()` (ver sección 5) que lanza un error legible apenas falta algo, en vez de que el problema aparezca varios pasos después como un `KeyError` sin contexto.

3. Dashboard — prototipo Streamlit

La aplicación se desarrolló en Streamlit y está organizada en un panel lateral de filtros globales y tres pestañas de visualización.

3.1 Controles del usuario

Todos los filtros del sidebar afectan en tiempo real al conjunto de datos visualizado:

Control	Tipo	Descripción
Rango de edad	Slider [0–100]	Restringe los registros por edad de la persona desaparecida
Estado de aparición	Selectbox	Filtra entre todos los casos, solo aparecidos o solo no aparecidos
Top N regiones	Slider [5–15]	Controla cuántos departamentos se muestran en el gráfico de barras
Nube de palabras sobre	Selectbox	Selecciona la columna de texto a analizar
Columna de texto para temas	Selectbox	Columna sobre la que se ejecuta el topic modeling (sección 6)
Número de temas (K)	Slider [2–8]	Cantidad de clusters para el K-Means del topic modeling
Columna de texto para sentimiento	Selectbox	Columna sobre la que se calcula el sentimiento (sección 7)
Tono del texto	Selectbox	Filtra el dashboard completo por Positivo / Neutral / Negativo / Todos

3.2 Pestaña "Explorar"

Histograma de edad por estado de aparición: muestra la distribución etaria de los casos segmentada por si la persona apareció o no. Permite observar que los menores de 18 años son el grupo mayoritario y que la distribución de aparecidos y no aparecidos es similar en todos los rangos de edad, sugiriendo que la edad no es un factor determinante en la resolución del caso.

Casos por mes: serie temporal con el recuento de denuncias por mes según la fecha del hecho. Permite identificar tendencias y posibles estacionalidades en los registros.

Tabla filtrada: vista tabular de los registros activos según los filtros aplicados, con opción de descarga en formato CSV.

3.3 Pestaña "Análisis"

Estado de aparición (donut): muestra la proporción global de casos resueltos frente a pendientes, actualizada dinámicamente con los filtros.

Distribución de horas hasta aparecer (box plot): aplicado únicamente sobre los casos resueltos, permite visualizar la dispersión del tiempo de resolución. La mediana se ubica alrededor de los 7 días, con una larga cola de casos que tardaron semanas o meses.

Top N departamentos: barras horizontales ordenadas de mayor a menor frecuencia. Lima domina el ranking, pero departamentos como Cusco y Arequipa presentan frecuencias más altas de lo que su peso poblacional sugeriría.

Mapa geográfico: distribución espacial de los casos sobre el mapa del Perú, con puntos de color verde para casos aparecidos y rojo para no aparecidos.

3.4 Pestaña "Texto"

Nube de palabras: generada sobre las columnas de texto libre del dataset. En **Circunstancias** los términos más frecuentes revelan los contextos más comunes de desaparición. En **Vestimenta** permiten construir un perfil rápido de la indumentaria reportada. El usuario puede alternar entre columnas desde el sidebar.

Temas (clustering): además de la nube de palabras, se agrupan los casos por similitud de texto usando TF-IDF + K-Means y se muestran los términos característicos de cada grupo, un gráfico de tamaño por tema y una proyección 2D de los clusters. El detalle de la metodología está en la sección 6.

Análisis de sentimientos: se clasifica cada caso en Positivo/Neutral/Negativo usando un modelo Transformer pre-entrenado en español (RoBERTa), con un gráfico comparando el tono por estado de aparición y descarga del resultado en CSV. El detalle de la metodología está en la sección 7.

4. Resultados Preliminares

Para obtener hallazgos iniciales, se revisaron tres escenarios dentro del dashboard: una vista general con personas de 0 a 100 años, una vista de menores de edad entre 0 y 17 años, y una vista de adultos entre 18 y 100 años. Esta comparación permite observar cómo cambian los indicadores principales según el rango etario seleccionado.

En la vista general, se registran 3,768 casos. De ellos, el 22.9% corresponde a personas reportadas como aparecidas y el 77.1% figura como no aparecida dentro de la base analizada. Además, los menores de 18 años

representan el 53.3% de los registros, lo que evidencia una presencia importante de población menor de edad en los reportes de desaparición.

Al comparar los grupos etarios, se observa que los menores concentran 2,008 casos, mientras que los adultos registran 1,760 casos. La proporción de personas aparecidas es similar en ambos grupos: 23.3% en menores y 22.4% en adultos. Sin embargo, sí se aprecia una diferencia en el tiempo hasta la denuncia: en menores, la mediana es de 18.5 horas, mientras que en adultos aumenta a 27.7 horas. Esto sugiere, de manera descriptiva, que los casos de menores tienden a ser denunciados con mayor rapidez.

En términos geográficos, Lima concentra la mayor cantidad de casos en los tres escenarios analizados. En la vista general también destacan departamentos como Cusco, Arequipa, Lambayeque, La Libertad, Junín, Puno, Huánuco, Piura y Áncash, lo que evidencia una concentración territorial marcada en determinados departamentos.

Finalmente, la nube de palabras construida a partir de la variable **Circunstancias** muestra términos frecuentes como "salió", "domicilio", "dirección", "casa", "desconocido", "paradero", "colegio", "menor" y "trabajo". Estos términos permiten identificar, de manera preliminar, contextos asociados a salidas del domicilio, lugares cotidianos y situaciones en las que se desconoce el paradero de la persona.

5. Decoradores y manejo de errores

Para la entrega final se centralizó el manejo de errores y se agregaron dos decoradores reutilizables en un nuevo módulo, **src/utils.py**, usado por **processing.py**, **viz.py** y **topics.py**.

5.1 Decoradores

@log_step envuelve las funciones de carga y procesamiento (**load_data**, **clean_data**, **get_summary** en **processing.py**, y **fit_topics** en **topics.py**). Registra el inicio y la duración de cada llamada y el shape del DataFrame resultante; si la función lanza una excepción, registra el traceback completo antes de relanzarla:

```
def log_step(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            result = func(*args, **kwargs)
        except Exception:
            logger.exception("%s' falló tras %.3fs", func.__name__,
                             time.perf_counter() - t0)
            raise
        logger.info("%s' OK en %.3fs", func.__name__, time.perf_counter() - t0)
        return result
    return wrapper
```

@safe_plot envuelve las nueve funciones de graficación de **viz.py**. Si una función falla -por ejemplo, porque tras filtrar el DataFrame quedó sin la columna esperada o sin filas-, en vez de propagar la excepción y tumbar todo el dashboard, devuelve una figura vacía con el error visible en el título:

```
def safe_plot(fallback_title="No se pudo generar el gráfico", as_none=False):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            try:
                return func(*args, **kwargs)
            except Exception as e:
                logger.exception("Error en '%s': %s", func.__name__, e)
                if as_none:
                    return None
                fig = go.Figure()
                fig.update_layout(title=f"{fallback_title}: {type(e).__name__}")
                return fig
        return wrapper
    return decorator
```

5.2 Manejo de errores

`require_columns(df, columnas, contexto)` valida, al inicio de cada función de `processing.py` y `topics.py`, que el DataFrame tenga las columnas necesarias antes de operar sobre él. Si falta alguna, lanza un `ValueError` con la lista de columnas faltantes en vez de dejar que el problema aparezca como un `KeyError` a mitad de una transformación.

`load_data()` además valida que el archivo exista (`FileNotFoundError`) y que el CSV sea legible (`ValueError` si está vacío o mal formado). En `app.py`, `get_data()` envuelve la carga completa en un `try/except`: si algo falla, se muestra el error con `st.error()` y se detiene la ejecución con `st.stop()`, en vez de mostrar al usuario un traceback completo de Python.

El mismo principio se aplicó al topic modeling: si se piden más temas (K) que casos con texto disponible tras los filtros aplicados, o si el vocabulario TF-IDF queda vacío, `fit_topics()` lanza un `ValueError` que `app.py` captura y muestra como un aviso (`st.info()`) dentro de la pestaña de Texto, sin afectar el resto del dashboard.

6. Topic modeling con clustering (TF-IDF + K-Means)

Como complemento a la nube de palabras, se agregó un análisis de topic modeling sobre las columnas de texto libre (`Circunstancias`, `Vestimenta`, `Otras observaciones`), implementado en `src/topics.py` con TF-IDF + K-Means de `scikit-learn`. Se eligió este enfoque -en vez de un modelo de tópicos probabilístico (LDA) o un embedding semántico- por ser determinístico, rápido en CPU y sin dependencias adicionales fuera de las que ya usa el proyecto.

6.1 Metodología

1. **Vectorización TF-IDF:** cada texto se convierte en un vector de pesos por palabra, excluyendo stopwords en español y descartando palabras que aparecen en menos de 2 documentos (`min_df=2`) o en más del 60% de ellos (`max_df=0.6`).
2. **K-Means:** agrupa los vectores en K clusters, con K controlado por el usuario desde el sidebar.

3. **Términos por tema:** para cada cluster se extraen las palabras con mayor peso TF-IDF promedio dentro de ese grupo, que se muestran como etiqueta del tema (ej. "Tema 2: domicilio, salió, colegio, casa").
4. **Silhouette score:** métrica de calidad del agrupamiento (rango -1 a 1, valores más altos indican clusters mejor separados), calculada para el K elegido.
5. **Proyección 2D:** los vectores TF-IDF se reducen a 2 dimensiones con TruncatedSVD (equivalente a LSA) para graficar los clusters en un scatter plot.

```
vectorizer = TfidfVectorizer(stop_words=STOPWORDS_ES_SIN_TILDE, max_df=0.6,
min_df=2)
X = vectorizer.fit_transform(textos)

kmeans = KMeans(n_clusters=n_topics, random_state=42, n_init=10)
cluster_ids = kmeans.fit_predict(X)

silhouette = silhouette_score(X, cluster_ids)
```

6.2 Resultados

Al aplicar el modelo sobre **Circunstancias** con K=4 (el valor por defecto del dashboard), se obtuvo un silhouette score de **0.015**. Es un valor bajo (el rango va de -1 a 1), lo que indica que los clusters no quedan bien separados entre sí. Esto es consistente con la naturaleza del texto: las descripciones de **Circunstancias** siguen un formato muy estandarizado entre denuncias ("salió de su domicilio...", "domicilio ubicado en..."), por lo que la mayoría de los vectores TF-IDF terminan siendo similares entre sí.

Esto se refleja en el tamaño de los temas: el Tema 4 concentra 3,241 de los 3,785 casos (85.6%), mientras que los otros tres son considerablemente más chicos:

- Tema 1 (113 casos): desconoce, paradero, momento, salió, actual, fecha, ese, domicilio
- Tema 2 (194 casos): domicilio, salió, desconocida, dirección, circunstancias, retorno, colegio, trabajo
- Tema 3 (232 casos): desconocido, rumbo, salió, domicilio, casa, dirección, vivienda, destino
- Tema 4 (3,241 casos): salió, domicilio, casa, dirección, encontraba, colegio, ubicado, centro

El Tema 4 funciona más como un grupo genérico que agrupa la redacción estándar de la mayoría de las denuncias. Los temas 1 a 3 capturan variaciones de vocabulario más específicas: el Tema 1 se asocia a casos donde se desconoce el paradero por un periodo de tiempo ("momento", "fecha", "actual"), el Tema 2 a salidas con un retorno esperado desde el colegio o el trabajo, y el Tema 3 a casos descritos con un rumbo o destino determinado. Se probó además con K entre 3 y 6, y el silhouette score se mantiene bajo en todos los casos (0.013 a 0.02), lo que sugiere que el texto no tiene una estructura natural de múltiples grupos bien diferenciados más allá de estas variaciones de vocabulario.

6.3 Limitaciones

K-Means asume clusters de forma aproximadamente esférica y de tamaño similar en el espacio TF-IDF, lo cual es una simplificación fuerte para texto libre. El número de temas se elige manualmente comparando el silhouette score entre distintos valores de K; un trabajo futuro razonable sería automatizar esa selección barriendo varios valores de K y quedándose con el que maximice la métrica.

7. Análisis de sentimientos (RoBERTuito / pysentimiento)

Sobre las mismas columnas de texto libre (**Circunstancias**, **Otras observaciones**) se implementó un análisis de sentimientos con un modelo Transformer pre-entrenado en español, usando la librería **pysentimiento**. A diferencia del topic modeling (sección 6), que agrupa documentos por similitud léxica sin supervisión, este análisis clasifica cada texto en una polaridad (positivo/neutral/negativo) con un modelo ya entrenado, sin ajustarlo sobre estos datos.

7.1 Modelo

Se usó **pysentimiento/robertuito-sentiment-analysis**: un modelo RoBERTa (RoBERTuito) pre-entrenado sobre más de 500 millones de tweets en español y ajustado para sentimiento con el corpus TASS 2020. Descrito en Pérez et al. (2021), "*pysentimiento: A Python Toolkit for Opinion Mining and Social NLP tasks*" (arXiv:2106.09462).

```
from pysentimiento import create_analyzer

analyzer = create_analyzer(task="sentiment", lang="es")
resultados = analyzer.predict(lista_de_textos)
```

Cada resultado trae una etiqueta (**POS/NEU/NEG**) y la probabilidad de cada clase. Se mapeó a Positivo/Neutral/Negativo, y se calculó un score continuo como $P(\text{POS}) - P(\text{NEG})$, en el rango [-1, 1].

7.2 Implementación

El modelo se carga una sola vez por sesión (**@st.cache_resource** en **app.py**, ya que **pysentimiento** depende de **torch** y **transformers** y tarda en cargar). Si el modelo no carga -sin internet la primera vez que se descarga, ~435 MB, o falta la librería-, la app muestra un aviso (**st.warning()**) y el resto del dashboard sigue funcionando sin esa sección, siguiendo el mismo principio de manejo de errores descrito en la sección 5.2.

La predicción se corre en batch sobre toda la columna de texto a la vez, no fila por fila, aprovechando el procesamiento por lotes de **transformers**.

7.3 Controles agregados al dashboard

Ver la tabla de la sección 3.1 ("Columna de texto para sentimiento" y "Tono del texto"). Este último filtra el dashboard completo, no solo la pestaña de Texto: al elegir "Negativo", por ejemplo, también cambian el histograma de edad, el mapa y los demás gráficos.

Los resultados se muestran en la pestaña "Texto": tres KPIs (% Positivo/Neutral/Negativo), un gráfico de barras del tono por estado de aparición, y un botón de descarga del CSV con el sentimiento calculado por caso.

7.4 Resultados

Sobre **Circunstancias**, la distribución de tono obtenida fue: 15.3% Negativo, 84.6% Neutral y 0.2% Positivo.

El sesgo hacia Neutral es muy marcado, y la clase Positivo es casi inexistente. Esto es consistente con la limitación señalada en 7.5: el registro de **Circunstancias** es factual y administrativo ("salió de su domicilio y

no regresó"), muy distinto al registro de los tweets con los que se entrenó el modelo, donde el sentimiento suele expresarse de forma explícita. El modelo parece reservar la etiqueta Positivo para un lenguaje abiertamente emocional que casi no aparece en este tipo de reporte, incluso en casos con desenlace favorable.

Al cruzar tono con estado de aparición, la proporción de casos "Apareció" es similar entre Negativo y Neutral -en ambos cerca del rango 23-25% reportado para el dataset completo en la sección 4-, por lo que el tono del texto no parece diferenciar el estado de aparición. La clase Positivo tiene muy pocos casos como para sacar una conclusión sobre ese grupo en particular.

7.5 Limitaciones

El modelo fue entrenado con tweets (TASS 2020), no con relatos de circunstancias de desaparición, por lo que clasifica con un vocabulario y un registro distintos a los de este dataset. Es un punto de partida razonable - entiende contexto y negación mejor que un enfoque de palabras clave-, pero no está ajustado a este dominio específico; un trabajo futuro sería fine-tunear el modelo con una muestra etiquetada manualmente de este mismo dataset.

`pysentimiento` y este modelo son de uso no comercial / investigación (heredan la licencia del corpus TASS), lo cual es compatible con este proyecto académico pero debe declararse.